

scikit-learn (sklearn) 速学笔记

0. 安装与环境

conda

```
conda install -c conda-forge scikit-learn pandas numpy matplotlib
```

或 pip

```
pip install -U scikit-learn pandas numpy matplotlib
```

版本检查:

```
import sklearn, numpy, pandas
print(sklearn.__version__)
```

1. sklearn 的核心理念

- 统一接口: `fit(X, y)`、`predict(X)`、`transform(X)`、`fit_transform(X)`、`score(X, y)`
- 组件化积木: **Estimator** (估计器, 模型/预处理器)、**Transformer** (变换器)、**Pipeline** (流水线)、**ColumnTransformer** (按列处理)
- 无状态 vs. 有状态: `StandardScaler` 在 `fit` 阶段学习均值方差, 在 `transform` 阶段应用

2. 快速上手范式 (通用模板)

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split,
cross_val_score
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
```

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score

# 1) 准备数据
df = pd.read_csv("data.csv")          # 行=样本, 列=特征/标签
y = df["label"]
x = df.drop(columns=["label"])

# 2) 划分集合
X_train, X_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42, stratify=y
)

# 3) 列类型
num_cols = x.select_dtypes(include=["number"]).columns
cat_cols = x.select_dtypes(include=["object", "category",
"bool"]).columns

# 4) 预处理 (数值标准化、类别独热)
pre = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
    ],
    remainder="drop",
)

# 5) 管道 + 模型
clf = Pipeline(steps=[
    ("preprocess", pre),
    ("model", LogisticRegression(max_iter=500))
])

# 6) 训练与验证
clf.fit(X_train, y_train)
pred = clf.predict(X_test)
proba = clf.predict_proba(X_test)[: , 1]
```

```
print(classification_report(y_test, pred))
print("ROC-AUC:", roc_auc_score(y_test, proba))
```

3. 常见任务与对应 API

3.1 分类 (Classification)

- 线性: `LogisticRegression`, `SGDClassifier`
- 树系: `DecisionTreeClassifier`, `RandomForestClassifier`, `GradientBoostingClassifier`
- 大杀器 (需安装) : `HistGradientBoostingClassifier` (sklearn 自带, 适合中大规模)
- 其他: `SVC`, `KNeighborsClassifier`, `NaiveBayes` (`GaussianNB`, `MultinomialNB`)

二分类样例 (含交叉验证) :

```
from sklearn.model_selection import cross_val_score,
StratifiedKFold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(clf, X, y, cv=cv, scoring="roc_auc")
print(scores.mean(), scores.std())
```

3.2 回归 (Regression)

- 线性: `LinearRegression`, `Ridge`, `Lasso`, `ElasticNet`
- 树系: `RandomForestRegressor`, `GradientBoostingRegressor`, `HistGradientBoostingRegressor`
- 其他: `SVR`, `KNeighborsRegressor`

回归评分: `r2_score`, `mean_absolute_error`, `mean_squared_error`

3.3 聚类 (Clustering)

- `KMeans`, `MiniBatchKMeans`
- `DBSCAN` (密度聚类)
- 评价 (无监督通常用内在指标) : `silhouette_score`

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

km = KMeans(n_clusters=3, n_init="auto", random_state=42)
labels = km.fit_predict(X) # 无 y
print("Silhouette:", silhouette_score(X, labels))
```

3.4 降维 (Dimensionality Reduction)

- `PCA`、`TruncatedSVD` (稀疏/文本)、`TSNE` (可视化)、`UMAP` (需额外安装)

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2).fit(X)
X2 = pca.transform(X)
```

4. 预处理与特征工程

- 数值: `StandardScaler`、`MinMaxScaler`、`PowerTransformer` (Box-Cox/Yeo-Johnson)、`PolynomialFeatures`
- 类别: `OneHotEncoder(handle_unknown="ignore")`
- 缺失值: `SimpleImputer(strategy="mean/most_frequent/constant")`, `KNNImputer`
- 列组合: `ColumnTransformer` (强烈推荐)
- 特征选择: `SelectKBest`, `RFE`, `SelectFromModel`

缺失值 + 多项式特征:

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import PolynomialFeatures
num_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("poly", PolynomialFeatures(degree=2, include_bias=False)),
    ("scale", StandardScaler()),
])
```

5. Pipeline 与 ColumnTransformer (实战关键)

- **好处**：把预处理与模型打包，保证交叉验证/部署时处理一致；避免数据泄漏
- **参数命名**：<step名>__<参数名>，用于网格搜索

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "model__C": [0.1, 1, 10],
    "model__penalty": ["l2"],
}

grid = GridSearchCV(clf, param_grid, cv=5, scoring="roc_auc",
n_jobs=-1)
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)
```

6. 模型评估与交叉验证

- 分类指标：accuracy, precision, recall, f1, roc_auc, average_precision (PR-AUC)
- 回归指标：r2, neg_mean_absolute_error, neg_root_mean_squared_error
- 交叉验证：cross_val_score, StratifiedKFold (分类)
- 学习曲线/验证曲线：learning_curve, validation_curve

```
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.metrics import RocCurveDisplay,
PrecisionRecallDisplay
ConfusionMatrixDisplay.from_estimator(clf, X_test, y_test)
RocCurveDisplay.from_estimator(clf, X_test, y_test)
PrecisionRecallDisplay.from_estimator(clf, X_test, y_test)
```

7. 不平衡数据要点

- 使用 `stratify` 分层抽样；关注 `ROC-AUC` 与 **PR-AUC**
- 类别权重: `class_weight="balanced"` (LogReg、LinearSVC、树系等支持)
- 采样方法 (需外部包 `imblearn`) : `SMOTE`, `RandomUnderSampler` (可与 `Pipeline` 结合)

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(class_weight="balanced", max_iter=500)
```

8. 文本特征与基本 NLP

- `CountVectorizer`, `TfidfVectorizer` 将文本→稀疏矩阵
- 与线性模型/朴素贝叶斯/线性 SVM 结合效果常常很好

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import SGDClassifier
from sklearn.pipeline import make_pipeline

clf_text = make_pipeline(
    TfidfVectorizer(ngram_range=(1,2), min_df=2),
    SGDClassifier(loss="log_loss", max_iter=1000)
)
clf_text.fit(text_train, y_train)
```

9. 时间序列注意事项（简单指引）

- 普通 `train_test_split` 会泄漏未来信息；使用按时间切分
- 交叉验证用 `TimeSeriesSplit`

```
from sklearn.model_selection import TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5)
```

- 许多时间序列模型不在 sklearn 内（如 ARIMA、Prophet），但特征法（滞后、滚动窗口统计）可用 sklearn 监督学习处理

10. 持久化与部署

- `joblib.dump/load` 保存 **完整 Pipeline**

```
import joblib
joblib.dump(clf, "clf.joblib")
loaded = joblib.load("clf.joblib")
```

- 与 FastAPI/Flask 集成：在服务启动时 `load`，预测时传入与训练一致的列名与类型

11. 常见陷阱与解决

- **数据泄漏**：在训练集上 `fit` 预处理器，只对验证/测试做 `transform`。用 `Pipeline` 一劳永逸
 - **类别新值**：`OneHotEncoder(handle_unknown="ignore")`
 - **收敛慢**：增大 `max_iter`，特征缩放，调学习率或正则
 - **随机性**：设置 `random_state` 复现实验
 - **性能**：优先 `HistGradientBoosting*`、`RandomForest*`；使用 `n_jobs=-1`；减少不必要的 `GridSearch` 维度
-

12. 迷你配方 (Cheat Sheet)

- 二分类首选起点:

```
Pipeline(pre -> LogisticRegression(class_weight="balanced"))
```

- 回归首选起点:

```
Pipeline(pre -> HistGradientBoostingRegressor())
```

- 类别多且稀疏:

```
OneHotEncoder(sparse_output=True, handle_unknown="ignore") + 线性模型
```

- 高维+非线性:

```
RandomForest/GradientBoosting + 合理调参
```

- 小数据防过拟合:

交叉验证 + 强正则 (C 小 α 大) + 简化特征

13. 两个端到端示例

13.1 二分类 (信用违约风格)

```
from sklearn.datasets import make_classification
X, y = make_classification(n_samples=3000, n_features=20,
                           n_informative=6,
                           weights=[0.8, 0.2], random_state=42)

df = pd.DataFrame(X, columns=[f"x{i}" for i in range(X.shape[1])])
df["catA"] = pd.qcut(df["x0"], 4, labels=
["Q1", "Q2", "Q3", "Q4"]).astype("category")
y = pd.Series(y, name="label")

X_train, X_test, y_train, y_test = train_test_split(
    df, y, test_size=0.2, stratify=y, random_state=42
)

num_cols = df.select_dtypes(include=["number"]).columns
cat_cols = df.select_dtypes(exclude=["number"]).columns
```



```

pre = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])

model = Pipeline([
    ("pre", pre),
    ("clf", LogisticRegression(class_weight="balanced",
max_iter=1000))
]).fit(X_train, y_train)

print(model.score(X_test, y_test))
print(classification_report(y_test, model.predict(X_test)))

```

13.2 回归（房价风格）

```

from sklearn.datasets import fetch_california_housing
from sklearn.metrics import mean_absolute_error
from sklearn.ensemble import HistGradientBoostingRegressor

X, y = fetch_california_housing(return_X_y=True, as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

reg = Pipeline([
    ("scale", StandardScaler(with_mean=False)), # HGBR 不强依赖，但
加速有时更稳
    ("model", HistGradientBoostingRegressor(random_state=42))
]).fit(X_train, y_train)

pred = reg.predict(X_test)
print("MAE:", mean_absolute_error(y_test, pred))

```